

Programming Assignment #2 — Indexer and Query Processor

Turi Rezende

Universidade Federal de Minas Gerais

Belo Horizonte, Brazil

<turi@ufmg.br>

1 Introduction

This project implements the indexer and query processor modules of a web search engine for a corpus of structured Wikipedia entity documents. The indexer receives a JSONL corpus, preprocesses each document, builds inverted lists under a fixed memory budget, writes the final index to disk, and reports summary statistics. The processor receives the generated index, preprocesses each query with the same pipeline used for documents, retrieves candidate documents with conjunctive document-at-a-time matching, scores them with either TF-IDF or BM25, and prints the top results in JSON format.

The implementation follows the main policies required in the specification:

- **Pre-processing policy:** documents and queries are lower-cased, tokenized, filtered with stopword removal, and stemmed with an English Snowball stemmer.
- **Memory management policy:** the indexer monitors process RSS with `psutil` and flushes partial indexes to disk when memory reaches a configurable fraction of the budget.
- **Parallelization policy:** document preprocessing is parallelized with a thread pool and a configurable number of worker threads.
- **Compression policy:** postings are stored with docid gaps and variable-byte integer encoding.
- **Query processing policy:** the processor uses the same preprocessor, conjunctive DAAT matching, and two ranking functions: TF-IDF and BM25.

The code is organized around small modules. `corpus.py` streams JSONL records; `preprocessing.py` implements tokenization, stopword removal and stemming; `memory.py` monitors memory usage; `partial_index.py`, `postings.py`, and `merge.py` write and merge partial inverted lists; `document_index.py`, `lexicon.py`, and `index_reader.py` represent the final on-disk structures; `daat.py` implements conjunctive matching; and `ranking.py` implements TF-IDF and BM25 scoring. I also used ChatGPT as an AI-based support tool to discuss implementation choices, debugging, and report organization; the final implementation and design decisions are my own.

2 Data Structures and Algorithms

The indexer uses a SPIMI-like workflow. The corpus is streamed line by line, avoiding loading the full JSONL file in memory. For each document, the selected textual fields are flattened into a single text, preprocessed, and converted into a local Counter of term frequencies. The in-memory partial index is a dictionary from terms to dictionaries of internal document ids and term frequencies: `dict[str, dict[int, int]]`. This representation gives average-case constant-time updates while documents are being processed.

When the memory monitor indicates that RSS reached the flush threshold, the current partial index is written to a sorted partial file. Each term record stores the UTF-8 term and an encoded posting list. Posting lists are represented as sorted pairs (d, tf) , encoded using docid gaps and variable-byte integers. After all documents have been processed, the partial files are merged externally. A heap keeps the smallest current term from each open partial file, groups equal terms, merges their posting lists, writes the merged list into `postings.bin`, and writes one lexicon entry into `lexicon.tsv`. The lexicon stores document frequency, collection frequency, byte offset, and byte length, which makes random access to each posting list possible. Document metadata are stored in `documents.bin`, including document lengths, external ids, number of documents, total tokens, and average document length.

The query processor loads the lexicon into memory and opens the postings and document index files. Each query is preprocessed with the same class used during indexing. If any preprocessed query term is absent from the lexicon, the conjunctive result is empty. Otherwise, query terms are sorted by increasing document frequency, their posting lists are read from disk, and the DAAT matcher advances all lists to the same candidate docid. Whenever all posting lists agree on a document, the scorer computes TF-IDF or BM25 and a small heap keeps only the top 10 results. Sorting query terms by document frequency reduces work because the shortest posting lists constrain the intersection first.

3 Complexity Analysis

Let N be the number of documents, L_i the number of raw tokens or characters scanned in document i , U_i the number of distinct indexed terms in document i , V the vocabulary size, P the total number of postings, and K the number of partial index files. Preprocessing and counting one document costs $O(L_i)$. Updating the in-memory partial index costs $O(U_i)$ on average because dictionary operations are average-case constant time. Therefore, the main in-memory indexing phase costs

$$T_{\text{index}} = O\left(\sum_{i=1}^N (L_i + U_i)\right)$$

plus the cost of flushing and merging partial files.

Writing a partial index is linear in the number of terms and postings in that partial, except for sorting terms and document ids. Across the whole run, encoding is $O(P)$. The external merge uses a heap over at most K open files, so the merge is approximately $O(P \log K)$ plus sequential disk I/O. The memory consumption of the indexer is bounded by the in-memory partial dictionary and worker batch size; the final document table and partial files are kept on disk. The processor uses $O(V)$ memory for the lexicon and reads posting lists on demand.

For a query with m indexed terms and posting-list lengths $p_1, \dots, p_m$, preprocessing is linear in the query length. Conjunctive DAAT costs at most linear in the scanned postings, with additional binary-search skips when a list is behind the current target docid. Each matched document is scored in $O(m)$ time, and maintaining the top-10 heap costs $O(\log 10)$, which is constant in practice. Thus, a useful upper bound for query processing is

$$T_{\text{query}} = O\left(\sum_{j=1}^m p_j + Mm\right),$$

where M is the number of conjunctively matched documents.

4 Index and Query Characterization

The full corpus contains 4,641,784 structured Wikipedia entity documents. The final index is stored in three files: `postings.bin`, containing the compressed posting lists; `lexicon.tsv`, containing one entry per indexed term; and `documents.bin`, containing the document-level statistics used during ranking. Table 1 summarizes the produced index.

Table 1: Summary of the produced index.

Statistic	Value
Indexed documents	4,641,784
Indexed tokens	195,413,164
Average document length	42.10
Inverted lists	2,407,292
Total postings	133,933,830
Average postings/list	55.64
Final index size (MiB)	454.47
Indexing time (s)	1,107.37
Indexing rate (docs/s)	4,191.71

The posting-list distribution is highly skewed, as shown in Table 2. The median posting list has size 1, meaning that at least half of the indexed terms occur in a single document. This behavior is expected in an entity corpus after stemming and stopword removal, because many terms correspond to rare names, specific entities, or uncommon expressions. At the same time, the maximum posting-list size is 1,147,087, showing that some general terms still occur in a large fraction of the collection. This skew is important for query processing: conjunctive DAAT can be very efficient when a query contains at least one selective term, but broader queries containing frequent terms require scanning many more candidate documents.

Table 2: Distribution of posting-list sizes.

Statistic	Posting-list size
Minimum	1
Median	1
90th percentile	9
95th percentile	23
99th percentile	262
Maximum	1,147,087

I evaluated the query processor using 50 test queries covering computer science, history, geography, science, music, sports, software, economics, and humanities topics. Each query was preprocessed with the same stopword removal and stemming policy used during indexing. Candidate documents were retrieved using conjunctive document-at-a-time matching and then scored with both TF-IDF and BM25. Table 3 lists the 50 evaluation queries, grouped by topic. Table 4 summarizes the query-processing results.

The number of matched documents is the same for TF-IDF and BM25 because matching is performed before ranking and uses the same conjunctive DAAT algorithm for both rankers. The smallest candidate set was obtained for the query `blockchain`, with only 17 matched documents, while the largest candidate set was obtained for `united states`, with 413,926 matched documents. This confirms that query cost is strongly affected by the document frequencies of the query terms. More specific technical queries, such as `deep learning`, `information retrieval`, and `linux kernel`, produced much smaller candidate sets, while broad entity queries such as `united states`, `world war`, and `association football` matched many more documents.

Table 5 characterizes the score distributions. For each query, I computed the minimum, mean, median, 90th percentile, 95th percentile, and maximum score over all matched documents. The table reports the average of these per-query statistics across the 50 queries.

The TF-IDF scores are generally larger and have a wider range than BM25 scores. This happens because BM25 applies term-frequency saturation and document-length normalization, so repeating a query term many times in a long document has a bounded effect compared with a more direct TF-IDF weighting. In this collection, the average maximum TF-IDF score across queries was 37.02, while the corresponding BM25 value was 18.26. The largest observed TF-IDF score was 57.02, for `world war ii`, while the largest BM25 score was 28.15. This suggests that BM25 produced a more compressed score scale, while still returning the same number of top results per query.

5 Empirical Efficiency

Table 6 reports the observed indexing throughput for different numbers of preprocessing threads. The approximate time is computed using the full corpus size of 4,641,784 documents. In this experiment, adding threads reduced throughput instead of improving it, with the best result obtained using one thread.

The final end-to-end run used to produce the submitted index processed 4,641,784 documents and 195,413,164 tokens in 1,107.37 seconds, reaching 4,191.71 documents per second. It produced 2,407,292 inverted lists, 133,933,830 postings, and a final index size of 454.47 MiB.

The negative thread scaling is probably caused by the CPU-bound nature of the pipeline: tokenization, stopword filtering, stemming, term-frequency counting, and dictionary updates. In CPython, threads are limited by the Global Interpreter Lock for Python bytecode execution, so more threads can add scheduling overhead and contention without true parallel speedup. A possible improvement would be to use multiple processes instead, since

Table 3: Queries used in the query-processing evaluation.

Category	Queries
Computer science and IR	information retrieval; search engine; computer science; database systems; operating systems; distributed systems; computer graphics; software engineering; algorithm design
Artificial intelligence	artificial intelligence; machine learning; deep learning; natural language processing; neural network; data mining
History, geography, and politics	world war; world war ii; united states; new york city; los angeles; european union
Science and environment	climate change; renewable energy; quantum mechanics; general relativity; molecular biology; cell biology; genetic engineering; human brain
Music, sports, and games	classical music; rock music; jazz music; electric guitar; basketball; association football; olympic games; video game
Software and technology	open source software; linux kernel; python programming; java programming; web development; cryptocurrency; bitcoin; blockchain
Humanities and social sciences	economics; philosophy; psychology; history of science; art history

Table 6: Indexing throughput by number of preprocessing threads.

Threads	Docs/s	Rel. speed	Approx. time (s)
1	5,978	1.00	776.5
2	4,690	0.78	989.7
4	3,824	0.64	1,213.9
8	3,437	0.57	1,350.5
16	3,145	0.53	1,475.9

Table 4: Query-processing workload over 50 queries.

Statistic	Value
Queries processed	50
Total matched documents	1,042,594
Mean matched docs/query	20,851.88
Median matched docs/query	2,225
Minimum matched docs/query	17
Maximum matched docs/query	413,926
Returned results per ranker	500
Elapsed time: TF-IDF (s)	7.81
Elapsed time: BM25 (s)	7.92

Table 5: Average per-query score distribution statistics.

Statistic	TF-IDF	BM25
Average minimum score	11.73	3.95
Average mean score	16.47	11.33
Average median score	15.01	11.08
Average 90th percentile	23.61	14.60
Average 95th percentile	25.91	15.52
Average maximum score	37.02	18.26
Global maximum score	57.02	28.15

Python's multiprocessing module side-steps the GIL by using subprocesses.